

GUÍA 2 PARA DESARROLLAR EN PLANTUML

Relación entre casos de uso, diagrama de clases de análisis y diagramas de secuencia de análisis

Caso base: Administrar Sistema

Elemento	Descripción
Asignatura	Análisis y Diseño de Software
Tema	Trazabilidad entre casos de uso, clases de análisis y secuencias de análisis con PlantUML
Caso base	Gestión de Autenticación y Seguridad con los datos Usuario, Contraseña, OTP, Correo
Operaciones CRUD	Iniciar Sesión, Cambiar Contraseña, Recuperar Contraseña y Autenticación Multifactor
Referencia base	01_Guia_PlantUML_CU_Analisis_CRUD_TekMess_V1.0

1. Propósito de la guía

Esta guía complementa el modelado inicial de casos de uso del sistema TekMess. Su finalidad es demostrar cómo el requisito funcional **RF-02: Administrar Sistema** (que permite controlar el acceso, validar identidades y proteger la información operativa de la plataforma) puede analizarse de forma progresiva mediante tres modelos técnicos conectados: casos de uso de análisis, clases de análisis y diagramas de secuencia de análisis.

El énfasis de este documento no es la programación de la interfaz, sino la comprensión de las responsabilidades de cada elemento del modelo (Frontera, Control y Entidad) y asegurar la trazabilidad completa desde la necesidad de seguridad expresada en la ERS hasta la interacción detallada entre los objetos del sistema.

A través de esta metodología, se busca garantizar que cada acción realizada por el usuario en el módulo de autenticación sea validada técnica y dinámicamente antes de su implementación en el entorno de desarrollo.

2. Relación general entre los tres modelos

Modelo	Pregunta que responde	Producto en PlantUML	Uso en la siguiente etapa
Casos de uso de análisis	¿Qué necesita hacer el Usuario para acceder de forma segura al sistema?	Diagrama de casos de uso y matriz de trazabilidad RF-CU	Permite identificar las responsabilidades del sistema y definir las clases frontera, control y entidad necesarias para cumplir con el RF-02.
Clases de análisis	¿Qué objetos participan internamente para autenticar o recuperar una clave?	Diagrama de clases con estereotipos: boundary (interfaz), control (lógica), entity (datos de sesión) y repository (persistencia en PostgreSQL)	Define los objetos y métodos específicos que aparecerán e interactuarán en los diagramas de secuencia.
Secuencias de análisis	¿Cómo colaboran el actor y los objetos para ejecutar el login o la verificación MFA?	Diagramas de secuencia por operación (Iniciar, Cambiar, Recuperar, Validar MFA)	Valida que el flujo de mensajes sea coherente con los requisitos de seguridad antes de la implementación en Dart/Flutter.

3. Caso base actualizado: Gestión de Empleados (TekMess)

El módulo se basa en una interfaz denominada "Acceso al Sistema", diseñada para proteger la entrada a la plataforma TekMess mediante credenciales y OTP.

Dato	Tipo sugerido	Descripción	Validación mínima
Nombre de Usuario	Texto	Identificador único generado por el sistema	Obligatorio, debe existir previamente en la BD.
Contraseña	Texto (Hash)	Clave de seguridad del usuario	Obligatorio, mínimo 8 caracteres (política de seguridad).
Código OTP	Texto	Código de verificación de un solo uso	Obligatorio, numérico, con expiración de 3 a 15 minutos.
Correo	Texto	Dirección electrónica para el envío del código OTP	Obligatorio, formato de email válido.

Botón / Acción de interfaz	Caso de uso asociado	Descripción de análisis
Ingresar	Iniciar Sesión	Captura credenciales, valida contra PostgreSQL y verifica estado del usuario.
Cambiar Clave	Cambiar Contraseña	Permite actualizar el Hash de la contraseña previa validación de políticas de seguridad.
Olvidé mi clave	Recuperar Contraseña	Genera un OTP, lo envía al correo y permite restablecer el acceso.
Verificar OTP	Autenticación MFA	Compara el código ingresado con el generado en la BD para autorizar el ingreso final.

4. Requisitos funcionales de nivel 0

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-02.01	El sistema debe permitir el inicio de sesión validando credenciales y estado.	Usuario	Iniciar Sesión
RF-02.02	El sistema debe permitir el cambio de contraseña voluntario u obligatorio.	Usuario	Cambiar Contraseña
RF-02.03	El sistema debe permitir recuperar la contraseña mediante un código temporal.	Usuario	Recuperar Contraseña
RF-02.04	El sistema debe requerir Autenticación Multifactor (MFA) para el acceso.	Usuario	Realizar Autenticación Multifactor

5. Diagrama de casos de uso de análisis

Este diagrama representa las funciones visibles para los usuarios dentro de TekMess. La validación de credenciales y la autenticación multifactor se modelan como comportamientos obligatorios mediante una relación de inclusión (<<include>>)

.Código PlantUML sugerido

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false
```

```

actor "Usuario" as Usuario
actor "Sistema" as Sistema

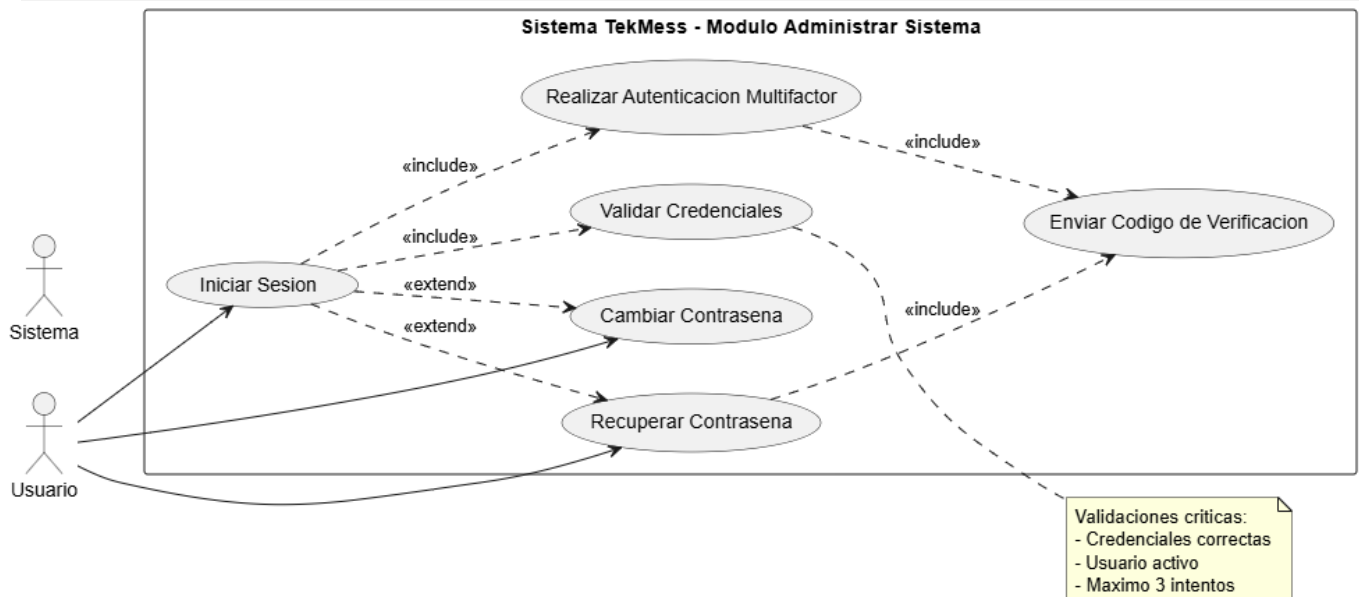
rectangle "Sistema TekMess - Modulo Administrar Sistema" {
    usecase "Iniciar Sesion" as UC1
    usecase "Cambiar Contraseña" as UC2
    usecase "Recuperar Contraseña" as UC3
    usecase "Realizar Autenticacion Multifactor" as UC4
    usecase "Validar Credenciales" as UC5
    usecase "EnviarCodigo de Verificacion" as UC6
}

Usuario --> UC1
Usuario --> UC2
Usuario --> UC3

UC1 ..> UC5 : <<include>>
UC1 ..> UC4 : <<include>>
UC4 ..> UC6 : <<include>>
UC3 ..> UC6 : <<include>>

UC1 ..> UC2 : <<extend>>
UC1 ..> UC3 : <<extend>>

note right of UC5
Validaciones criticas:
- Credenciales correctas
- Usuario activo
- Maximo 3 intentos
end note
@enduml
    
```



6. Derivación de clases de análisis

A partir de los casos de uso del módulo RF-02: Administrar Sistema, identificamos las clases de análisis utilizando los estereotipos de frontera, control, entidad y repositorio.

Esta separación asegura que la lógica de seguridad esté aislada de la interfaz y de la base de datos PostgreSQL.

Tipo de clase	Responsabilidad en el análisis	Clase propuesta	Relación con el CRUD
Boundary / Frontera	Recibe datos del Usuario y muestra resultados de la autenticación.	FormularioAccesoSistema	Representa la pantalla con campos para Usuario, Contraseña y Código OTP.
Control	Coordina la lógica de login, encriptación y validación MFA.	ControlAutenticacion	Valida políticas de contraseña, verifica códigos OTP y gestiona accesos.
Entity / Entidad	Representa la información de sesión en el sistema.	Usuario, CodigoVerificacion	Contiene los atributos críticos: hash de contraseña, estado, rol y códigos temporales.
Repositorio / Colección	Conserva y recupera los datos de acceso en la base de datos.	RepositorioUsuario, RepositorioCodigoVerificacion	Permite buscar usuarios, actualizar claves y validar expiración de OTP.

Código PlantUML del diagrama de clases de análisis

```
@startuml

skinparam classAttributeIconSize 0
skinparam shadowing false
left to right direction

class "FormularioGestionEmpleado" <<boundary>> {
    - txtCedula: String
    - txtNombres: String
    - txtCorreo: String
    - cbRol: RolRBAC
    + clickRegistrar()
    + clickActualizar()
    + clickDarBaja()
    + mostrarMensaje(msg: String)
    + listarPersonal(empleados: List)
}

class "ControlEmpleado" <<control>> {
    + crearEmpleado(datos): Resultado
    + editarEmpleado(cedula, datos): Resultado
    + darBajaEmpleado(cedula): Resultado
    + validarDatos(cedula, correo): boolean
    + generarCredenciales(): void
}

class "Empleado" <<entity>> {
    - cedula: String
```

```

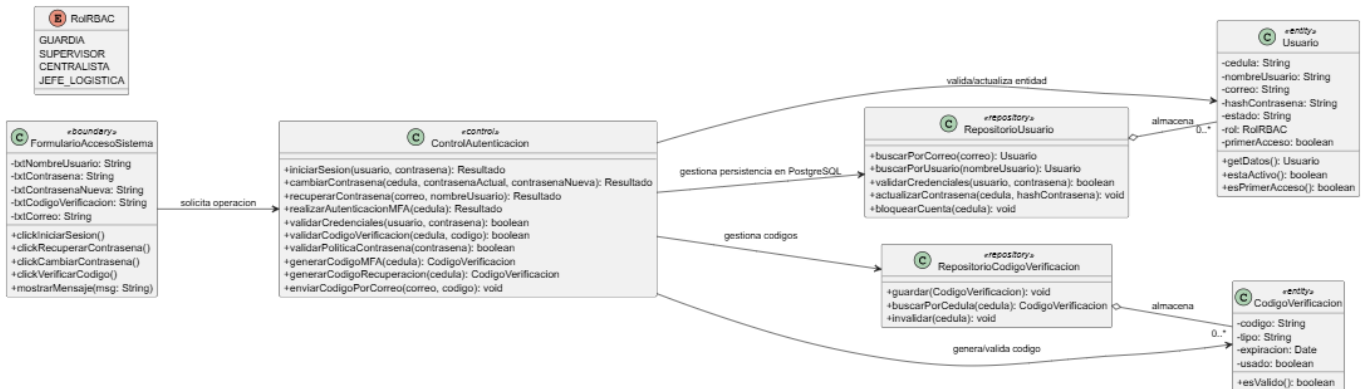
- nombres: String
- rol: RolRBAC
- correo: String
+ getDatos(): Empleado
}

enum "RolRBAC" {
    GUARDIA
    SUPERVISOR
    CENTRALISTA
    JEFE_LOGISTICA
}

class "RepositorioEmpleado" <<repository>> {
    + existeCedula(cedula): boolean
    + guardar(Empleado): void
    + buscarPorCedula(cedula): Empleado
    + actualizar(Empleado): void
    + eliminar(cedula): void
}

FormularioGestionEmpleado --> ControlEmpleado : solicita operación
ControlEmpleado --> Empleado : crea/actualiza entidad
ControlEmpleado --> RepositorioEmpleado : gestiona persistencia en PostgreSQL
RepositorioEmpleado o-- "0..*" Empleado : almacena

@enduml
    
```



7. Matriz de relación: casos de uso, clases y secuencias

Caso de uso	Boundary	Control	Entity / Repositorio	Diagrama de secuencia esperado
Iniciar Sesión	Captura credenciales.	Valida credenciales e invoca MFA.	Lee la entidad Usuario en RepositorioUsuario.	Ingresar datos -> Validar DB -> Verificar Estado -> Llamar MFA -> Conceder acceso.

Cambiar Contraseña	Carga formulario de cambio de claves.	Valida política de seguridad y aplica Hash.	Actualiza el Hash en RepositorioUsuario.	Ingresar claves -> Validar políticas -> Actualizar -> Confirmar.
Recuperar Contraseña	Solicita correo de recuperación.	Genera OTP, lo envía y coordina el cambio.	Crea CodigoVerificacion y lo guarda en RepositorioCodigo.	Solicitar -> Generar OTP -> Enviar Correo -> Validar OTP -> Cambiar clave.
Autenticación MFA	Captura código OTP ingresado.	Compara el OTP ingresado con el almacenado.	Verifica expiración en RepositorioCodigoVerificacion.	Generar OTP -> Enviar -> Ingresar código -> Validar -> Acceso exitoso.

8. Diagramas de secuencia de análisis

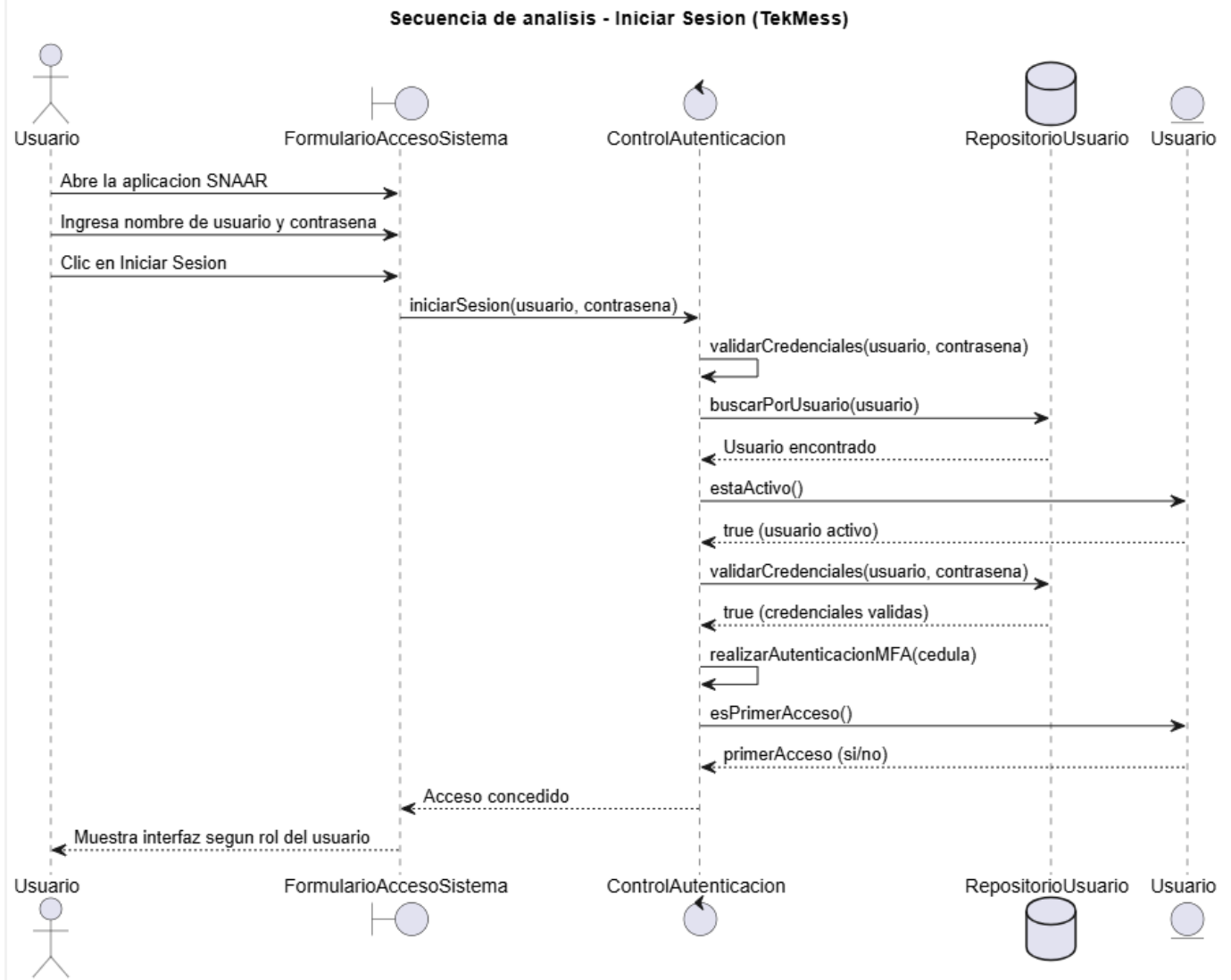
Estos diagramas muestran la colaboración dinámica de los objetos de TekMess a lo largo del tiempo para cumplir con las operaciones de seguridad del RF-02.

8.1 Secuencia: Iniciar Sesión

```
@startuml
title Secuencia de analisis - Iniciar Sesion (TekMess)

actor "Usuario" as Usuario
boundary "FormularioAccesoSistema" as UI
control "ControlAutenticacion" as Ctrl
database "RepositorioUsuario" as RepoU
entity "Usuario" as User

Usuario -> UI : Abre la aplicacion SNAAR
Usuario -> UI : Ingresa nombre de usuario y contrasena
Usuario -> UI : Clic en Iniciar Sesion
UI -> Ctrl : iniciarSesion(usuario, contrasena)
Ctrl -> Ctrl : validarCredenciales(usuario, contrasena)
Ctrl -> RepoU : buscarPorUsuario(usuario)
RepoU --> Ctrl : Usuario encontrado
Ctrl -> User : estaActivo()
User --> Ctrl : true (usuario activo)
Ctrl -> RepoU : validarCredenciales(usuario, contrasena)
RepoU --> Ctrl : true (credenciales validas)
Ctrl -> Ctrl : realizarAutenticacionMFA(cedula)
Ctrl -> User : esPrimerAcceso()
User --> Ctrl : primerAcceso (si/no)
Ctrl --> UI : Acceso concedido
UI --> Usuario : Muestra interfaz segun rol del usuario
@enduml
```



8.2 Secuencia: Cambiar Contraseña

```

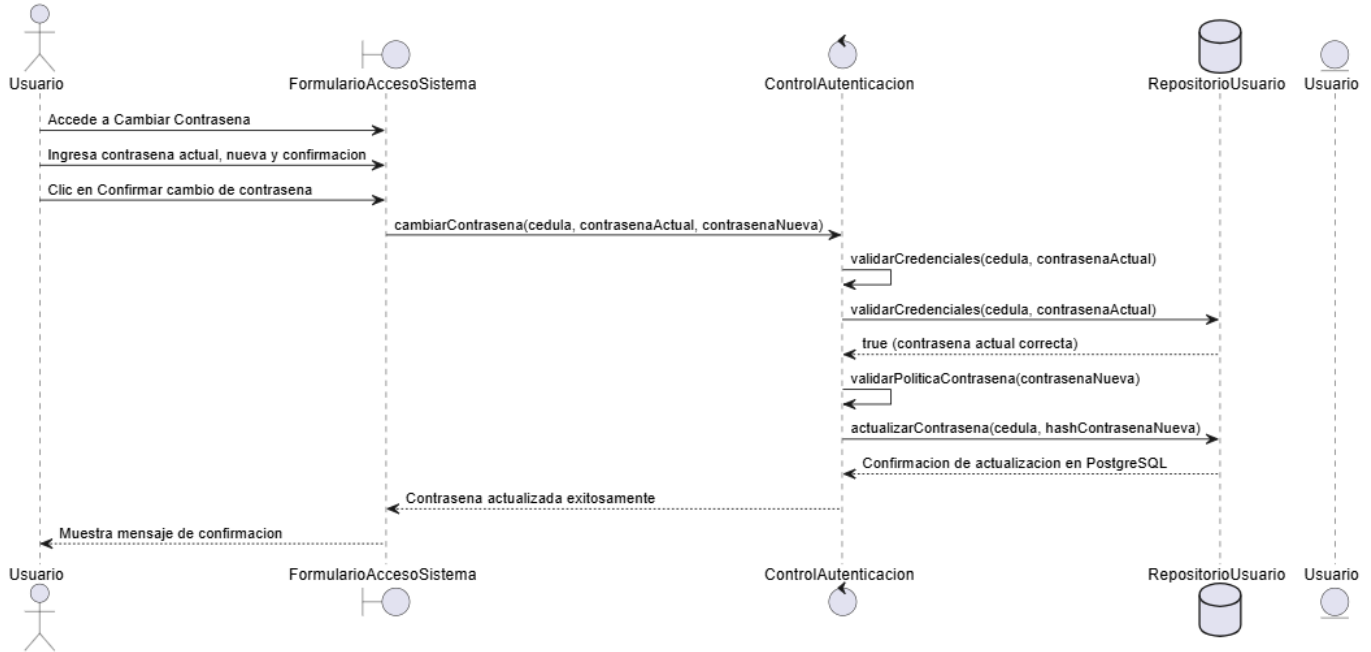
@startuml
title Secuencia de analisis - Cambiar Contraseña (TekMess)

actor "Usuario" as Usuario
boundary "FormularioAccesoSistema" as UI
control "ControlAutenticacion" as Ctrl
database "RepositorioUsuario" as RepoU
entity "Usuario" as User

Usuario -> UI : Accede a Cambiar Contraseña
Usuario -> UI : Ingresa contraseña actual, nueva y confirmacion
Usuario -> UI : Clic en Confirmar cambio de contraseña
UI -> Ctrl : cambiarContraseña(cedula, contraseñaActual, contraseñaNueva)
Ctrl -> Ctrl : validarCredenciales(cedula, contraseñaActual)
Ctrl -> RepoU : validarCredenciales(cedula, contraseñaActual)
RepoU --> Ctrl : true (contraseña actual correcta)
    
```

```
Ctrl -> Ctrl : validarPoliticaContrasena(contrasenaNueva)
Ctrl -> RepoU : actualizarContrasena(cedula, hashContrasenaNueva)
RepoU --> Ctrl : Confirmacion de actualizacion en PostgreSQL
Ctrl --> UI : Contraseña actualizada exitosamente
UI --> Usuario : Muestra mensaje de confirmacion
@enduml
```

Secuencia de analisis - Cambiar Contraseña (TekMess)



8.3 Secuencia: Recuperar Contraseña

```
@startuml

title Secuencia de analisis - Recuperar Contraseña (TekMess)

actor "Usuario" as Usuario
boundary "FormularioAccesoSistema" as UI
control "ControlAutenticacion" as Ctrl
database "RepositorioUsuario" as RepoU
database "RepositorioCodigoVerificacion" as RepoC
entity "Usuario" as User
entity "CodigoVerificacion" as Codigo

Usuario -> UI : Selecciona "Olvido su contraseña?"
Usuario -> UI : Ingresa correo y nombre de usuario
Usuario -> UI : Clic en Enviar codigo de recuperacion
UI -> Ctrl : recuperarContrasena(correo, nombreUsuario)
Ctrl -> RepoU : buscarPorUsuario(nombreUsuario)
RepoU --> Ctrl : Usuario encontrado
Ctrl -> Codigo : generarCodigoRecuperacion(cedula)
Ctrl -> RepoC : guardar (Codigo)
```

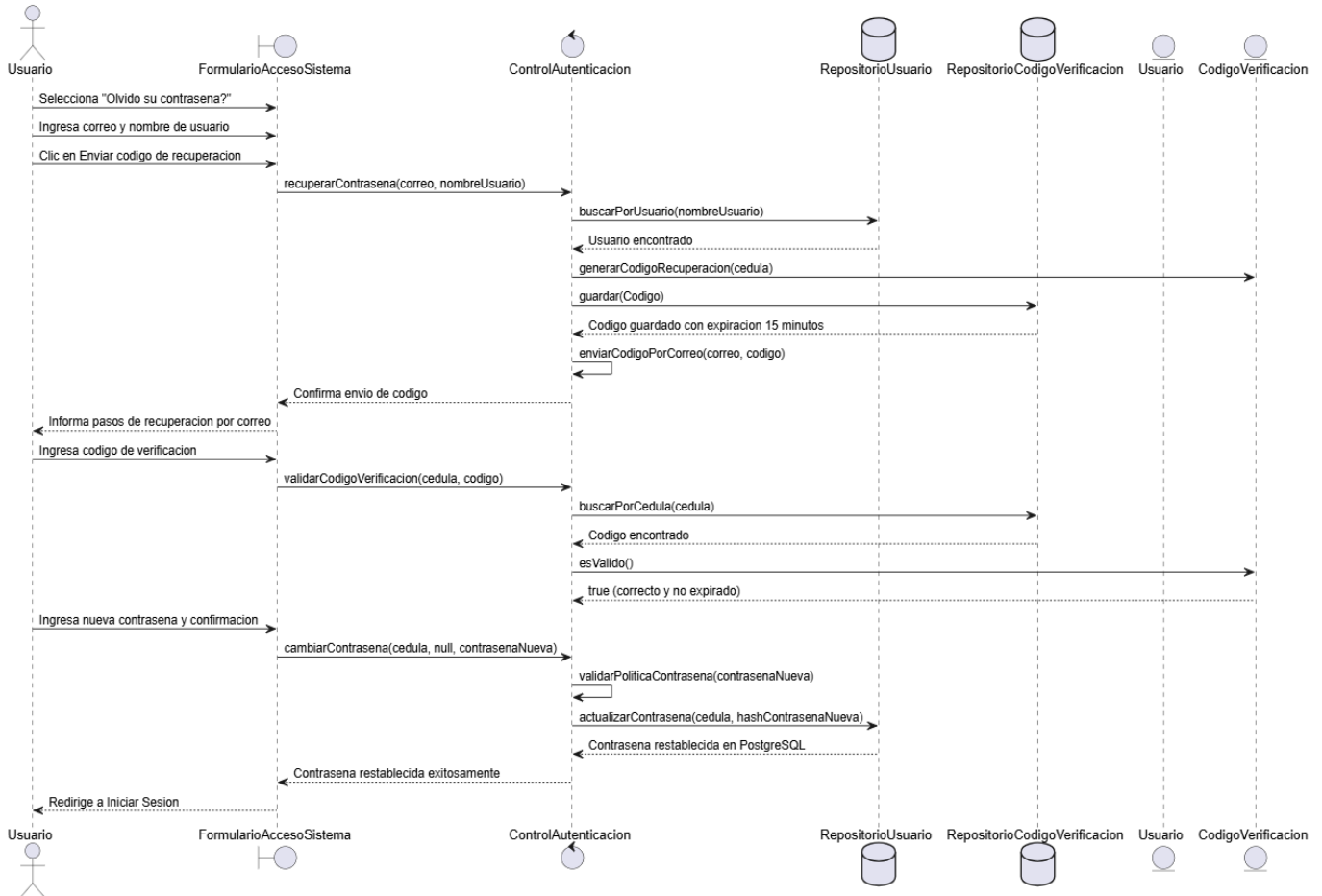


```

RepoC --> Ctrl : Código guardado con expiracion 15 minutos
Ctrl -> Ctrl : enviarCodigoPorCorreo(correo, codigo)
Ctrl --> UI : Confirma envio de codigo
UI --> Usuario : Informa pasos de recuperacion por correo
Usuario -> UI : Ingresa codigo de verificacion
UI -> Ctrl : validarCodigoVerificacion(cedula, codigo)
Ctrl -> RepoC : buscarPorCedula(cedula)
RepoC --> Ctrl : Código encontrado
Ctrl -> Código : esValido()
Código --> Ctrl : true (correcto y no expirado)
Usuario -> UI : Ingresa nueva contraseña y confirmacion
UI -> Ctrl : cambiarContraseña(cedula, null, contraseñaNueva)
Ctrl -> Ctrl : validarPoliticaContraseña(contraseñaNueva)
Ctrl -> RepoU : actualizarContraseña(cedula, hashContraseñaNueva)
RepoU --> Ctrl : Contraseña restablecida en PostgreSQL
Ctrl --> UI : Contraseña restablecida exitosamente
UI --> Usuario : Redirige a Iniciar Sesión

@enduml
    
```

Secuencia de analisis - Recuperar Contraseña (TekMess)

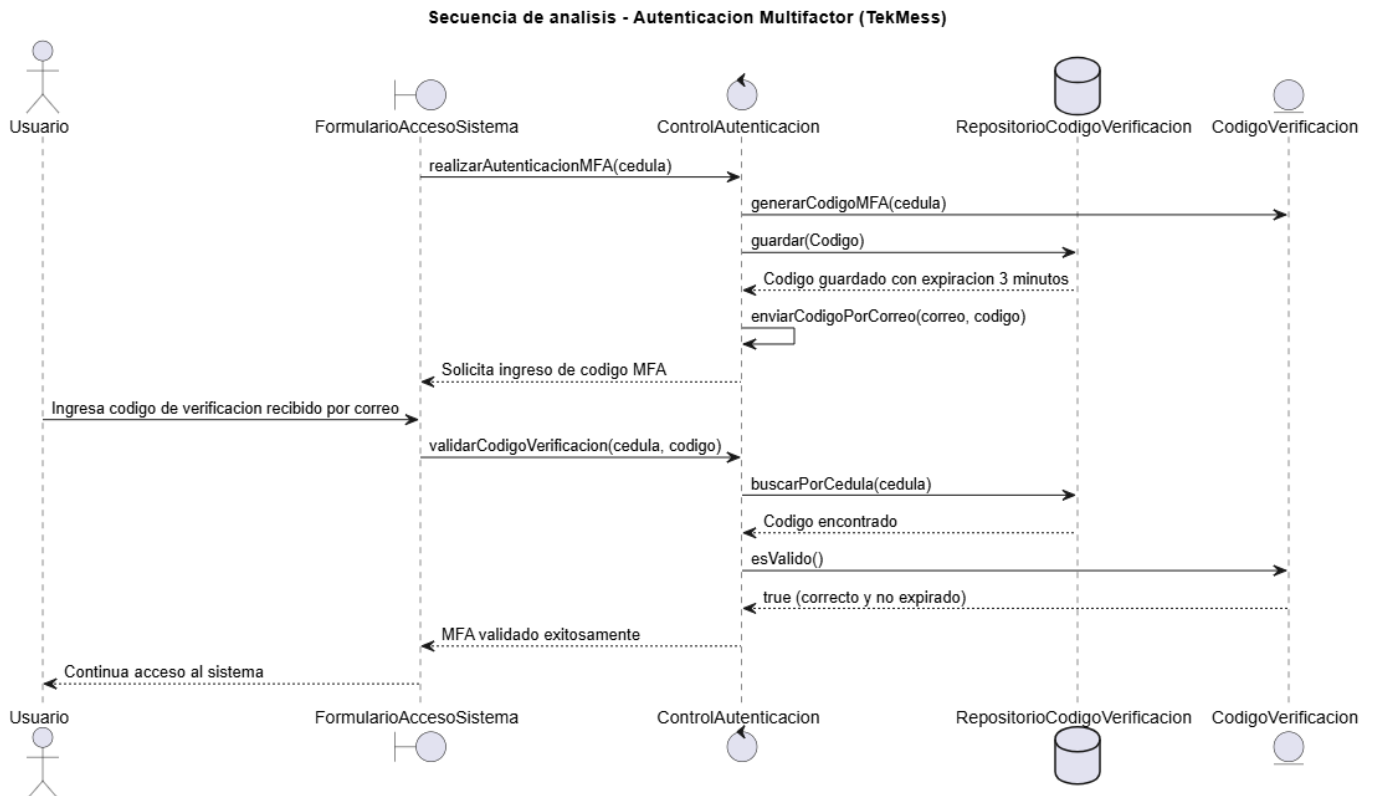


8.4 Secuencia: Autenticación Multifactor

```
@startuml
title Secuencia de analisis - Autenticacion Multifactor (TekMess)

actor "Usuario" as Usuario
boundary "FormularioAccesoSistema" as UI
control "ControlAutenticacion" as Ctrl
database "RepositorioCodigoVerificacion" as RepoC
entity "CodigoVerificacion" as Codigo

UI -> Ctrl : realizarAutenticacionMFA(cedula)
Ctrl -> Codigo : generarCodigoMFA(cedula)
Ctrl -> RepoC : guardar(Codigo)
RepoC --> Ctrl : Codigo guardado con expiracion 3 minutos
Ctrl -> Ctrl : enviarCodigoPorCorreo(correo, codigo)
Ctrl --> UI : Solicita ingreso de codigo MFA
Usuario -> UI : Ingresa codigo de verificacion recibido por correo
UI -> Ctrl : validarCodigoVerificacion(cedula, codigo)
Ctrl -> RepoC : buscarPorCedula(cedula)
RepoC --> Ctrl : Codigo encontrado
Ctrl -> Codigo : esValido()
Codigo --> Ctrl : true (correcto y no expirado)
Ctrl --> UI : MFA validado exitosamente
UI --> Usuario : Continua acceso al sistema
@enduml
```



9. Especificación breve de casos de uso conectada con las secuencias

Caso de uso	Precondición	Flujo principal resumido	Resultado esperado
Iniciar Sesión	El usuario debe estar registrado y activo en la BD.	Ingresar credenciales -> Validar formato -> Verificar DB -> Requerir MFA -> Conceder acceso	Acceso permitido al sistema con los permisos del RoIRBAC.
Cambiar Contraseña	El usuario debe conocer su contraseña actual.	Solicitar cambio -> Ingresar claves -> Validar políticas (8 caracteres) -> Aplicar Hash -> Actualizar en repositorio	Contraseña actualizada correctamente en PostgreSQL.
Recuperar Contraseña	El correo debe estar vinculado a un usuario válido.	Ingresar correo -> Generar OTP -> Enviar OTP -> Validar OTP ingresado -> Cambiar clave	Acceso restablecido mediante código temporal.
Realizar Autenticación Multifactor	Las credenciales primarias debieron ser correctas.	Iniciar proceso MFA -> Generar OTP -> Enviar por correo -> Ingresar código -> Validar expiración	Identidad confirmada como capa adicional de seguridad.

10. Conclusión técnica

La implementación de la metodología de análisis para el módulo Administrar Sistema (RF-02) de TekMess demuestra que la relación entre los casos de uso, las clases de análisis y los diagramas de secuencia garantiza una arquitectura de seguridad robusta. En este proceso de modelado, los casos de uso definen qué acciones de acceso requiere el usuario; las clases de análisis (Frontera, Control, Entidad y Repositorio) aseguran que la validación de contraseñas y códigos OTP no ocurra directamente en la interfaz; y los diagramas de secuencia ilustran cómo se verifica la identidad dinámicamente antes de conceder permisos. Esta trazabilidad técnica asegura que el diseño cumpla con los estándares de seguridad (como la Autenticación Multifactor y el encriptado de claves), integrándose correctamente con PostgreSQL antes de iniciar la codificación en Dart/Flutter. Finalmente, este enfoque protege los datos sensibles de la operación, minimizando el riesgo de accesos no autorizados al sistema de emergencias.